

Proyecto Final Segundo Cincuenta “Requisitos y Diagramas”

Presentado por: Juan Felipe Rodriguez Salazar

Presentado a: Ingeniero Jairo Riaño

Electiva 2: Desarrollo Web

VIGILADA MINEDUCACIÓN

Universidad Pedagógica y Tecnológica de Colombia

Sogamoso, Boyacá

2026

Requisitos Funcionales (RF)

Tabla 1. Módulo de Gestión de Mesas (Floor Plan)

Campo	Detalle
ID	RF-01
Nombre	Visualización y Apertura de Mesas
Actor	Mesero / Cajero
Prioridad (MoSCoW)	Must Have (Debe tener)
Descripción	El sistema debe mostrar un mapa interactivo con el estado de las mesas en tiempo real mediante un código de colores (Verde: Libre, Rojo: Ocupada, Amarillo: Pendiente). Al interactuar con una mesa libre, el sistema debe inicializar un nuevo ticket (<i>Guest Check</i>).
Criterio de Aceptación	El estado de la mesa debe actualizarse inmediatamente en la vista general cuando se cambia el estatus de la orden mediante el endpoint PUT <code>/api/orders/:id/status</code> .

Tabla 2. Módulo de Operaciones de Caja (POS)

Campo	Detalle
ID	RF-02
Nombre	Gestión de Comandas y Anulaciones (Void)
Actor	Mesero / Cajero



Prioridad (MoSCoW)	Must Have (Debe tener)
Descripción	El sistema debe permitir agregar productos al ticket activo desde un catálogo categorizado. Asimismo, debe permitir seleccionar un ítem específico del ticket y eliminarlo (Void), recalculando automáticamente el subtotal y el total antes de enviar la orden a cocina.
Criterio de Aceptación	Si se anulan todos los ítems de una mesa que ya había sido guardada en la base de datos, el sistema debe cambiar automáticamente el estado de la mesa a "Anulada" y liberarla en el mapa.

Tabla 3. Módulo de Transacciones y Pagos

Campo	Detalle
ID	RF-03
Nombre	Procesamiento de Pago y Cierre de Mesa
Actor	Cajero
Prioridad (MoSCoW)	Must Have (Debe tener)
Descripción	El sistema debe proporcionar una interfaz de cobro (Numpad) que soporte la introducción del monto entregado y múltiples métodos de pago (Efectivo, Tarjeta de Crédito, Débito, Gift Card). Al procesar exitosamente el pago, el ticket debe cerrarse.
Criterio de Aceptación	La confirmación del pago debe ejecutar una petición al backend que reduzca dinámicamente el stock del inventario para cada producto contenido en el ticket.

Tabla 4. Módulo de Administración de Catálogo



Campo	Detalle
ID	RF-04
Nombre	Creación de Productos Multitenant
Actor	Administrador de Tienda
Prioridad (MoSCoW)	Should Have (Debería tener)
Descripción	El administrador debe poder registrar nuevos productos en el catálogo especificando nombre, categoría, precio y stock base a través de un panel de control (Modal).
Criterio de Aceptación	Cada producto creado debe quedar estrictamente vinculado al tenantId del usuario autenticado, garantizando que el catálogo sea exclusivo y aislado para esa tienda.

Requisitos No Funcionales (RNF)

Tabla 5. Seguridad y Control de Acceso

Campo	Detalle
ID	RNF-01
Atributo de Calidad	Seguridad
Prioridad (MoSCoW)	Must Have (Debe tener)
Descripción	Todas las rutas de la API que gestionan inventario y transacciones financieras deben estar protegidas mediante autenticación basada en JSON Web Tokens (JWT).

Restricción Técnica	El token JWT debe generarse al iniciar sesión, almacenarse temporalmente en el cliente y adjuntarse en las cabeceras (Authorization: Bearer <token>) de cada petición HTTP.
----------------------------	---

Tabla 6. Arquitectura y Despliegue

Campo	Detalle
ID	RNF-02
Atributo de Calidad	Disponibilidad y Escalabilidad
Prioridad (MoSCoW)	Must Have (Debe tener)
Descripción	El sistema debe operar bajo una arquitectura en la nube que garantice alta disponibilidad, minimizando los tiempos de inactividad durante los picos de operación del restaurante.
Restricción Técnica	La solución debe desplegarse utilizando una plataforma de funciones Serverless (Vercel) para el Backend/Frontend, y un servicio gestionado de bases de datos (MongoDB Atlas) para la persistencia.

Tabla 7. Interfaz y Experiencia de Usuario

Campo	Detalle
ID	RNF-03
Atributo de Calidad	Usabilidad

Prioridad (MoSCoW)	Must Have (Debe tener)
Descripción	La interfaz gráfica debe estar optimizada para entornos operativos de alta velocidad, operando como una Single Page Application (SPA) para evitar recargas completas del navegador que interrumpen el flujo de caja.
Restricción Técnica	El diseño debe emplear un esquema de colores de alto contraste en modo oscuro (Dark Mode), optimizando la visibilidad en terminales táctiles (Touchscreens) y reduciendo la fatiga visual del cajero.

Tabla 8. Estructura y Mantenibilidad del Código

Campo	Detalle
ID	RNF-04
Atributo de Calidad	Mantenibilidad
Prioridad (MoSCoW)	Should Have (Debería tener)
Descripción	El código fuente del backend debe seguir estándares de modularidad que permitan incorporar futuras integraciones (ej. Impresión de comandas en cocina o facturación electrónica) sin alterar el núcleo del sistema.
Restricción Técnica	El backend debe estructurarse estrictamente bajo el patrón arquitectónico MVC (Model-View-Controller) sobre el framework Express.js.

DIAGRAMAS:

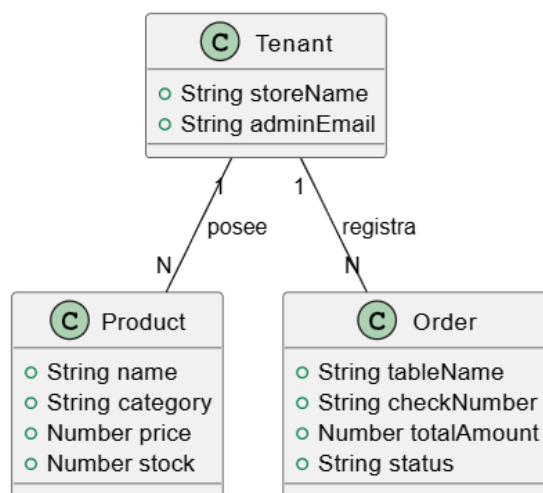


Imagen 1. 1. Vista Lógica (Clases)

Esta vista define los requisitos funcionales del sistema desde la perspectiva del usuario final. El diagrama ilustra las interacciones principales del actor (Mesero/Cajero) con el Sistema POS, abarcando el ciclo de vida completo de una transacción en piso: la apertura de mesas, el registro de comandas, la anulación de ítems (Void) por excepciones operativas y la ejecución final del pago para liberar la mesa. Estos escenarios validan el comportamiento esperado de la aplicación.

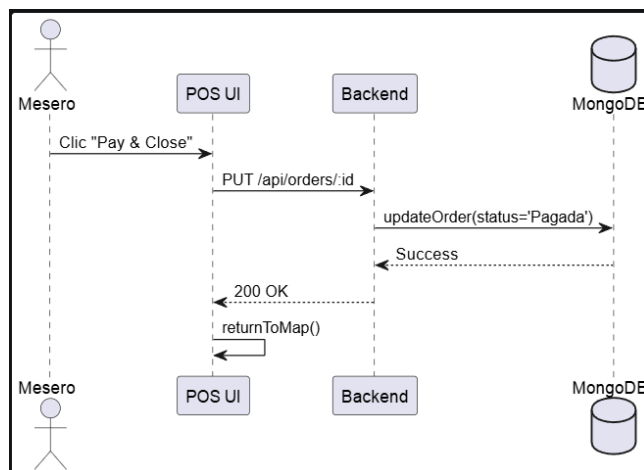


Imagen 2. Vista de Procesos (Secuencia)

El diagrama representa la estructura estática del sistema y el modelo de dominio mediante un enfoque de diseño Orientado a Objetos. Se destaca la entidad Tenant, fundamental para la arquitectura multitenant, la cual garantiza el aislamiento de datos entre diferentes tiendas. Esta entidad mantiene una relación de uno a muchos con las entidades Product (inventario) y Order (tickets o mesas), centralizando la gestión comercial de cada entorno.

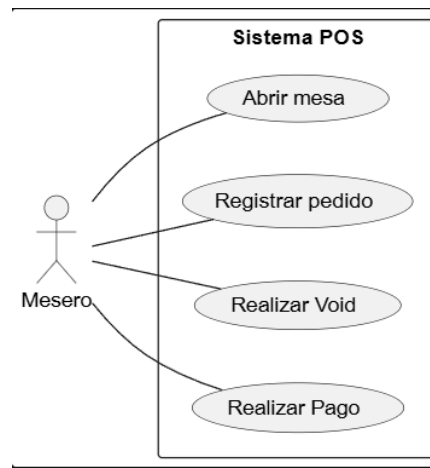


Imagen 3. Vista de Escenarios (+1) (Casos de Uso)

Esta vista detalla el comportamiento dinámico del sistema durante la ejecución del proceso crítico de "Pago y Cierre de Mesa". El diagrama traza el flujo de mensajes sincrónicos y asincrónicos entre la Interfaz de Usuario (POS UI), el servidor backend (API REST) y la base de datos (MongoDB). Se evidencia la persistencia del cambio de estado a "Pagada" y la posterior recarga del mapa de mesas en el cliente tras recibir el código de estado 200 OK, demostrando el control de concurrencia y flujo de datos.

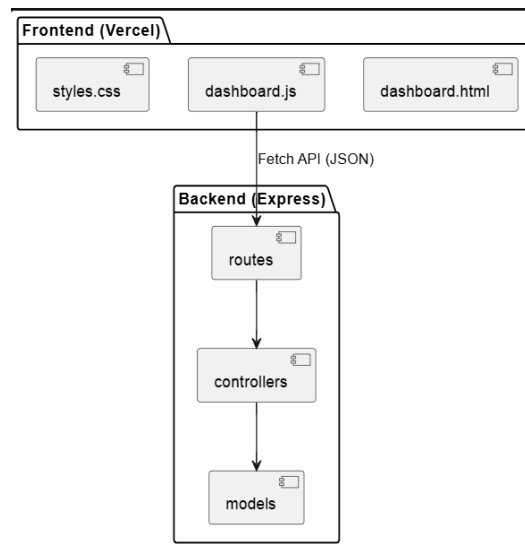


Imagen 4. Vista de Implementación (Componentes)

El diagrama ilustra la organización física de los módulos de software en el entorno de desarrollo. Se evidencia una arquitectura separada por responsabilidades: un paquete Frontend que contiene la lógica de la Interfaz Gráfica de Usuario (HTML, CSS, JS Vainilla) y un paquete Backend estructurado bajo el patrón de diseño MVC (Rutas, Controladores y Modelos en Express.js). La comunicación entre ambas capas se realiza estrictamente a través de peticiones HTTP consumiendo una API REST.

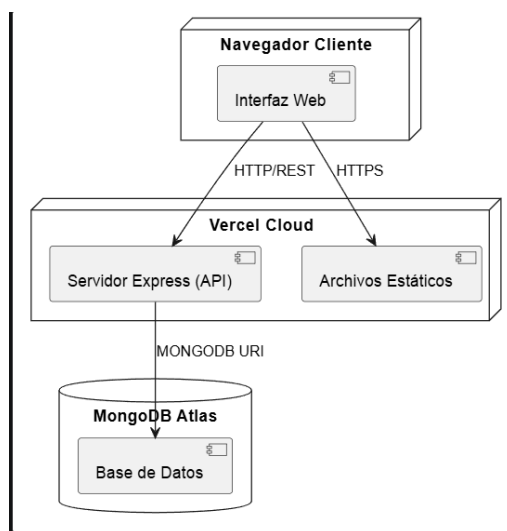


Imagen 5. Vista de Despliegue

Esta vista mapea la arquitectura de software sobre la infraestructura de hardware y servicios en la nube. El cliente (navegador web) interactúa de forma segura vía HTTPS con los servidores alojados en la plataforma *serverless* de Vercel, la cual sirve tanto los archivos estáticos como la ejecución dinámica de la API en Node.js. A su vez, la persistencia de los datos se delega a un clúster gestionado en MongoDB Atlas, garantizando alta disponibilidad y escalabilidad para la aplicación.